Частини 1–2: архітектура Uklon і батч на AWS

OOAD, лабораторна 8 · timlim24 · github.com/timlim24/OOAD-lab-8

part1-architecture/README.md

Частина 1 — Архітектура даних Uklon: шлях однієї поїздки

1. Що станеться, якщо real-time дорога відстане на 10 хвилин? А batch — на добу?

Real-time відстала на 10 хвилин → це інцидент, а не тікет

Streaming-дорога (IoT Core → Kinesis → Flink → Redis/DynamoDB) лежить **у критичному шляху продукту**. Відставання на 10 хвилин означає, що:

Компонент	Що саме ламається
Redis (позиції водіїв)	Матчинг бачить, де водій був 10 хв тому. Призначить водія, який уже за 5 км звідти або вже везе іншого пасажира. Пасажир бачить ETA 3 хв, водій їде 15.
DynamoDB (surge)	Множник порахований на попиті десятихвилинної давнини. Пік уже почався — ціна ще базова, водії не отримують сигналу їхати в район; пік закінчився — ціна ще підвищена, пасажири відвалюються. Прямий недозаробіток в обидва боки.
Flink (алерти)	Алерт про аномалію приходить тоді, коли аномалія вже 10 хвилин триває.
SNS (пуші)	«Водія знайдено» приходить після того, як пасажир уже скасував.
Трекінг поїздки	Пасажир бачить машину не там, де вона є.

Ключове: **дані, що протухли на 10 хвилин, гірші за відсутні дані**. Якщо Redis порожній, матчинг може чесно відповісти «водіїв немає». Якщо Redis повертає застарілі координати — система впевнено ухвалює неправильні рішення. Це сторінка для чергового інженера о третій ночі.

Окремо: Kinesis має retention (за замовчуванням 24 год), тож дані не губляться — Flink «дожене». Але відновлені через 10 хвилин surge-множники вже нікому не потрібні: вікно бізнес-цінності цих даних — секунди.

Batch відстала на добу → це тікет, а не інцидент

Batch-дорога (Firehose → S3 → Glue → Athena → QuickSight) лежить **у критичному шляху рішень, а не продукту**. Додаток працює далі: поїздки замовляються, водії призначаються, гроші списуються. Відставання на добу означає:

- дашборди PM-ів показують позавчорашні цифри — рішення відкладаються на день;

- нічний Glue job не відпрацював → наступний має обробити вдвічі більше даних (ризик каскаду: другий теж не встигне до ранку);
- SageMaker перетренується на день пізніше — моделі ETA і попиту продовжують обслуговувати трафік, просто на трохи старіших вагах. Деградація непомітна.

Але є винятки, де доба болить: - **виплати водіям**, якщо вони рахуються нічним батчем — водій не отримав гроші вчасно, це вже репутаційна і юридична проблема; - **антифрод-звіти** та регуляторна звітність із дедлайнами; - **реконсиляція з платіжним провайдером** — розбіжності знаходять на добу пізніше.

Коротко

Real-time дорога відповідає на питання «що робити **зараз**», batch — «що ми зробили **вчора**». Протухла відповідь на перше питання безглузда, на друге — просто запізніла. Тому SLO в них різні на три порядки: секунди проти годин.

---

2. Чому GPS не йде через Lambda?

Чотири незалежні причини, кожної окремо достатньо.

2.1 Економіка: 580 млн інвокацій на добу

580 млн пінгів/добу ≈ **6 700 подій/с** у середньому, і це середнє — у вечірній пік реально у 2–3 рази більше. За місяць це ≈ **17,4 млрд** подій.

Груба оцінка за прайс-листом us-east-1:

Стаття	Ціна	На місяць
Lambda requests	$0,20 / 1 млн	17 400 × $0,20 ≈ **$3 480**
Lambda duration (128 МБ, 20 мс)	$0,0000000021 / GB-ms	≈ **$900**
API Gateway REST перед Lambda	$3,50 / 1 млн	≈ **$60 900**
API Gateway HTTP API	$1,00 / 1 млн	≈ **$17 400**

Сам транспорт коштує більше, ніж усе інше в архітектурі разом. Саме про це рядок у завданні: «580 млн пінгів на добу через API Gateway коштували б десятки тисяч доларів на місяць».

Для порівняння: IoT Core бере ≈ $0,08 за 1 млн повідомлень (до 5 КБ) + $0,08 за 1 млн хвилин підключення. Kinesis — $0,015 за shard-годину; 7 шардів покривають 7 МБ/с, тобто ≈ **$75/місяць** за шард-години плюс $0,014 за 1 млн PUT-юнітів. Різниця на два порядки.

2.2 Протокол: пінг — це 50 байтів на постійному з’єднанні

GPS-пінг — крихітне повідомлення раз на 3 секунди. HTTP-запит до Lambda означає кожні 3 секунди: DNS → TCP handshake → TLS handshake → HTTP-заголовки → 50 байтів корисного навантаження. **Накладні витрати у 20–40 разів перевищують сам payload.**

На телефоні водія це ще й радіо: модем не встигає заснути між запитами → батарея сідає за зміну. MQTT тримає **одне постійне з’єднання** на водія з keep-alive, пінг летить як

кілька десятків байтів у вже відкритому сокеті.

2.3 Модель масштабування: Lambda рахує конкурентність, а не throughput

Lambda масштабується за кількістю **одночасних** виконань: `concurrency = rate × duration`. При 6 700 подій/с і 20 мс на подію це 134 одночасні інвокації — начебто нормально. Але:

- дефолтний ліміт акаунта — 1 000 одночасних виконань **на весь регіон**, тобто GPS з'їдає бюджет, якого не вистачить решті сервісів під час піку;
- burst на ранковому піку → **cold starts**, а cold start у 300 мс на пінгу, що має дійти за 3 секунди, — це вже лаг;
- throttling під час сплеску означає **втрачені пінги** без буфера, який їх притримає.

IoT Core тримає мільйони MQTT-з'єднань і його rule engine кладе повідомлення прямо в Kinesis. Kinesis — це **буфер**: Flink читає з нього пачками по сотні-тисячі записів за раз. Вартість і накладні витрати на один запис падають у сотні разів.

2.4 Стан: обробка GPS інкрементальна, а Lambda stateless

Щоб із пінгу зробити щось корисне, треба попередній стан водія: минула координата (щоб порахувати швидкість і напрямок), поточна поїздка, чи не стоїть водій у заторі, map-matching на дорожній граф. Lambda стану не має → **читання + запис у DynamoDB на кожен пінг**. Це 580 млн RCU+WCU на добу — знову десятки тисяч доларів, і ще й латентність мережевого походу на кожну подію.

Flink тримає **keyed state** у пам'яті воркера, розшардований за `driverId`. Доступ до попередньої позиції — це звернення до локального RocksDB/heap, наносекунди. Плюс Flink гарантує порядок подій у межах ключа і exactly-once через чекпоінти — Lambda не дає ні того, ні іншого.

Де Lambda в цій діаграмі доречна

У кроці 5: **одна Lambda на новий файл у S3**. Firehose складає тисячі подій у файл на 128 МБ раз на кілька хвилин → це ~300 інвокацій на добу замість 580 мільйонів. Той самий сервіс, але на правильній гранулярності: **подія-файл, а не подія-запис**. Це і є правило — Lambda хороша на рідких подіях із великим payload і погана на частих подіях із крихітним.

---

3. Що маленький стартап викинув би першим і що він на цьому втратить?

Порядок викидання — від найдорожчого з найменшою віддачею на малому масштабі.

Викидається першим

1. Apache Flink (Kinesis Data Analytics) Замість нього — surge рахується прямо в сервісі матчингу через лічильник у Redis (`INCR` на замовлення в районі, ключ із TTL 5 хв). *Втрата:* справжніх віконних обчислень за event time. Пізня подія або не врахується взагалі, або поламає лічильник. Немає exactly-once, немає реплею з початку стріму при зміні формули.

Логіка ціноутворення переїжджає в HTTP-хендлер, де її ніхто не тестує окремо. На 500 поїздок на день це прийнятно; на 50 000 — ні. *Чому першим:* Flink-кластер коштує сотні доларів на місяць незалежно від навантаження, а на малих обсягах лічильник у Redis дає 80% цінності за 0% ціни.

2. SageMaker *Втрата:* ML повністю. ETA рахується евристикою (відстань по прямій × коефіцієнт звивистості / середня швидкість у місті), прогнозу попиту немає — водії роз'їжджаються інтуїтивно. *Чому:* моделі ETA потребують місяців історії, щоб перемогти евристику. У стартапу цієї історії просто немає. Інвестиція без даних — це витрата.

3. Glue + Athena + QuickSight *Втрата:* дешева аналітика по історії. Замінник — SQL по read-replica Aurora і Metabase зверху. *Чому:* працює до десятків мільйонів рядків. Далі аналітичний запит на репліці починає з'їдати IOPS, реплікаційний лаг росте, і одного дня важкий звіт PM-а деградує прод. Це класичний момент, коли стартап і будує собі дата-лейк.

4. DMS change data capture *Втрата:* гарантії, що жодна зміна в БД не загубилася. Замінник — сервіс сам пише подію в стрім після коміту. *Чому боляче пізніше:* це **проблема подвійного запису**. Сервіс закомітив транзакцію в Aurora і впав до публікації події → у Aurora поїздка завершена, у дата-лейку ні. Розбіжності накопичуються тихо, знаходяться через півроку під час реконсиляції, і полагодити історію вже неможливо. CDC читає WAL — якщо воно в базі, воно і в стрімі. Правильна заміна, якщо CDC не по кишені, — **transactional outbox**: писати подію в таблицю тією ж транзакцією, окремий воркер її вичитує.

5. IoT Core *Втрата:* дешевий транспорт для мільйонів постійних з'єднань. Замінник — звичайний WebSocket на тих самих ECS-інстансах. *Чому:* на 100 водіях це пара мегабайтів трафіку. На 100 000 доводиться самому робити те, що IoT Core дає з коробки: автентифікація по сертифікатах, обробка обривів, шардування з'єднань по інстансах, backpressure.

Викидається останнім або ніколи

ALB + ECS + Aurora — це сам продукт, без нього немає сервісу.

S3 і Firehose — і це найважливіше. Сире складання подій у S3 коштує **копійки** ($0,023 за ГБ-місяць) і є єдиним кроком, який **неможливо зробити заднім числом**. Кластер Flink можна підняти завтра. Модель — натренувати через рік. Але дані за минулий рік, які ніхто не зберіг, не існують назавжди.

> Правило: **викидай обробку, не викидай збереження.** Обчислення можна додати пізніше, історію — ні.

Що з цього стосується нашої лабораторної

Частина 3 (Redpanda локально) — це і є «стартаповий» варіант: той самий стрім подій, але на одній машині і за нуль доларів. Частина 4 (Kinesis + Firehose + Athena) — те, у що він виростає, коли одна машина перестає тягнути. Різницю між ними ми міряємо в частині 4.

Частина 2 — Batch на AWS: платежі в S3

Як запустити

```
aws login                     # сесія протухає — без неї нічого не працює
./setup.sh                    # бакет + копія даних + Athena + sql.rendered/
./setup.sh crawler            # додатково: IAM-роль і Glue Crawler (Шлях Б)
```

`setup.sh` підставляє ім’я вашого бакета в усі запити й кладе результат у `sql.rendered/`. Далі виконуєте їх в Athena по порядку.

Файл	Що робить
sql/00_setup.sql	база `lab8`
sql/01_create_table.sql	зовнішня таблиця `payment_events` + `MSCK REPAIR`
sql/02_dedup_view.sql	в’ю `payments` — один рядок на `invoiceId`
sql/03_turnover.sql	оборот по мерчантах і валютах
sql/04_failure_rate.sql	частка відмов по методу й пристрою
sql/05_peak_hours.sql	платежі по годинах доби
sql/06_late_events.sql	запізнілі події
sql/07_ctas_parquet.sql	CTAS у Parquet
sql/08_parquet_queries.sql	запити 3–5 по Parquet
sql/09_scale_and_dedup.sql	цифри масштабу + демонстрація дедуплікації

> **Усі числа нижче — фактичні, з Athena.** Спершу пораховані локально через DuckDB, потім перевірені запитами в Athena по всьому датасету (31 файл, 1 260 672 849 Б). Збіглися до останньої цифри. Статистика кожного запуску — у results/benchmark.csv разом із `QueryExecutionId`.

Масштаб датасету

Показник	Значення
Усього подій	**3 030 197**
Унікальних `invoiceId`	**1 000 000**
Точних дублікатів (зайвих рядків)	**30 197** — 0,997 %
Подій після `DISTINCT`	3 000 000 — рівно 3 на платіж
Подій, що запізнилися > 60 с	**60 824** — 2,007 %
Подій, що запізнилися > 1 години	**0** — 0,0000 %
Максимальне запізнення	600 с = рівно 10 хвилин
Медіанне запізнення	1 с

Три відповіді на питання завдання:

1. **Унікальних `invoiceId` — 1 000 000.** Рівно мільйон, як і заявлено. Кожен платіж дав **рівно 3** події (`created → processing → success|failure`), а не «2–3»: перевірено, розподіл вироджений.
2. **Дублікатів — 30 197 (0,997 %).** Кожна продубльована подія прийшла рівно **двічі** — оригінал плюс один ретрай, трійок немає.
3. **Запізнілих більш ніж на годину — жодної (0 %).** Це не «запит не відпрацював»: генератор обмежує затримку десятьма хвилинами, і хвіст обривається рівно на 600 с.

Розподіл запізнення — він бімодальний, а не плавний

Інтервал	Подій	Частка
0–10 с	2 969 260	97,989 %
10–60 с	113	0,004 %
60–120 с	6 735	0,222 %
2–5 хв	20 137	0,665 %
5–10 хв	33 952	1,120 %

Практичний висновок для частини 3 — скільки подій уже є на руках при різному grace period:

grace	покриття
0 с	32,65 %
30 с	97,99 %
60 с	97,99 %
120 с	**98,22 %**
300 с	98,88 %
600 с	100,00 %

Дволвилинний grace із завдання ловить 98,2 % подій. Останні 1,8 % коштують ще вісім хвилин затримки на **кожному** алерті — класичний компроміс повнота/латентність.

Чим партиціоновано: event time чи processing time?

```
подій, де день modifiedDate ≠ dt :  0
подій, де день receivedAt    ≠ dt : 88
```

`dt` — це **день події**, не день отримання. 88 подій сталися перед північчю, а доїхали після. Це правильний вибір для аналітики: партиція за добу закривається назавжди і денний звіт ніколи не змінюється заднім числом. Якби партиціонували за `receivedAt`, учорашня партиція дописувалася б усю ніч і кожна денна цифра була б попередньою.

Запит 2 — дедуплікація

В'ю `lab8.payments` повертає **1 000 000 рядків**, `count(DISTINCT invoiceId)` — теж 1 000 000.

Що прибрав кожен крок:

```
3 030 197  сирих подій
-   30 197  ← SELECT DISTINCT       (передоставлені копії, 0,997 %)
= 3 000 000
- 2 000 000  ← row_number() = 1     (застарілі статуси created/processing)
= 1 000 000  платежів
```

Статуси після дедуплікації:

status	платежів	%
success	939 820	93,98
failure	60 180	6,02

Жоден платіж не застряг у `created`/`processing` — усі дійшли до термінального стану. У проді так не буває, і запит 4 має другий варіант саме на цей випадок.

Запит 3 — оборот по мерчантах і валютах (тільки успішні)

30 рядків: 10 мерчантів × 3 валюти. Повний вивід — у results/q3_turnover.md. Гривневі обороти:

Мерчант	Платежів	Оборот, ₴
Інтернет-магазин	87 115	310 259 259,21
АЗС	87 560	144 641 341,06
Квитки	87 303	77 749 863,65
Супермаркет	87 283	54 147 597,02
Доставка їжі	87 820	46 044 018,26
Аптека	87 374	38 554 151,04
Таксі	87 307	21 528 219,39
Онлайн-кінотеатр	87 357	13 016 193,00
Кав'ярня	87 637	11 435 687,68
Музичний сервіс	87 517	8 664 183,00

Кількість платежів у всіх мерчантів однакова з точністю до 0,6 % — датасет генерований рівномірно. Відрізняється **середній чек**: 3 562 ₴ в інтернет-магазині проти 99 ₴ у музичного сервісу. Тобто рейтинг «хто важливіший» повністю визначається чеком, а не трафіком.

Чому `finalAmount`, а не `amount`: `amount` — це скільки **запросили**, `finalAmount` — скільки реально **списали**, і він нульовий у всьому, що не `success`. Сума по `amount` порахувала б гроші, які не рухалися.

Валюти не складаються: 980, 840 і 978 — три різні одиниці, для зведення потрібен курс на дату, якого в датасеті немає.

Запит 4 — частка відмов по методу оплати й пристрою

paymentMethod	device	платежів	відмов	failure rate
qr	android	6 675	448	**6,71 %**
apple_pay	android	83 677	5 130	6,13 %
google_pay	web	59 996	3 645	6,08 %
card	android	183 332	11 079	6,04 %
apple_pay	ios	83 164	5 025	6,04 %
google_pay	ios	59 761	3 590	6,01 %
card	web	183 435	11 020	6,01 %
qr	web	6 669	400	6,00 %
google_pay	android	59 450	3 563	5,99 %
card	ios	183 541	10 912	5,95 %
apple_pay	web	83 505	4 966	5,95 %
qr	ios	6 795	402	**5,92 %**

Чесне прочитання цієї таблиці: різниці немає. Розкид 5,92–6,71 п.п. виглядає як сигнал, але для комірки на 6 675 платежів 95 % довірчий інтервал біноміальної частки при p = 0,06 становить ±0,57 п.п. Верхній рядок `qr/android` лежить якраз на межі, а всі решта — глибоко всередині шуму. Правильна відповідь на «який метод найгірший» тут — **жоден**, і це важливіше, ніж назвати переможця таблиці, відсортованої за шумом.

Причини відмов розподілені так само рівномірно (по ~12 000 кожна, 20 %):

failureReason	n
Таймаут 3-D Secure	12 107
Перевищено ліміт	12 088
Відмова банку-емітента	12 025
Недостатньо коштів	12 006
Невірний CVV	11 954

Запит 5 — платежі по годинах доби

год UTC	платежів		год UTC	платежів
00	6 597		12	66 304
01	6 622		13	59 948
02	6 592		14	53 306
03	6 629		15	53 783
04	**6 585**		16	60 256
05	13 291		17	72 905
06	26 851		18	**80 338**
07	46 684		19	73 167
08	59 947		20	60 219
09	53 492		21	46 627
10	46 521		22	26 669
11	53 438		23	13 229

Форма: рівна нічна підлога ~6 600/год з 00:00 до 04:00 UTC, ранковий підйом, обідній горб о 12:00 UTC і справжній пік о **18:00 UTC = 21:00 за Києвом** (80 338). Відношення пік/провал — **12,2×**.

Ці 12× — головна причина, чому в архітектурі Uklon усе на Fargate і Kinesis: залізо під піковий трафік простоювало б 2/3 доби.

Увага на таймзону: Київ у жовтні 2026 — UTC+3 до 25 жовтня і UTC+2 після, тож один фіксований зсув для місяця неправильний. У sql/05_peak_hours.sql є варіант через `AT TIME ZONE 'Europe/Kyiv'`, який це враховує.

Запит 7 — Parquet: наскільки дешевше

Розміри фактичні, після CTAS в Athena:

	JSON Lines	Parquet + Snappy (Athena)	Parquet (локально, DuckDB)
Байтів	1 260 672 849	**88 309 777**	79 080 980
Файлів	31	**213**	31
Рядків	3 030 197	1 000 000	1 000 000
		у 14,3× менше	у 15,9× менше

Parquet від Athena на 11,7 % більший за локальний, бо Athena пише **по файлу на воркер** — 6–7 файлів на партицію замість одного. Менші файли → менші row group-и → гірший словник і гірше стиснення. Це мініатюра **small files problem**: тут вона коштує 11 % місця, на 200 000 файлів коштувала б більше, ніж самі дані.

Тут складені **два різні ефекти**, і у звіті їх варто розділяти:

- **дедуплікація**: 3 030 197 → 1 000 000 рядків, це 3,03×. Її дав би й CTAS у JSON;
- **кодування**: решта ~5,2× — колонкове зберігання прибирає повторення імен ключів у кожному рядку, dictionary-кодує 10 назв мерчантів / 4 методи / 3 пристрої, бітпакує цілі.

Розмір по колонках показує, куди пішли байти:

колонка	MiB	частка
reference	30,71	41 %
invoiceId	10,49	14 %
createdDate	6,22	8 %
receivedAt	6,22	8 %
modifiedDate	6,22	8 %
customerId	5,43	7 %
amount	3,94	5 %
finalAmount	3,82	5 %
merchantName / merchantId / mcc	по 0,49	2 %
paymentMethod / device	по 0,25	0,7 %
ccy / failureReason / status	0,09–0,12	0,4 %

41 % файлу — це `reference`, випадковий UUID без дефісів. Він не стискається (ентропія максимальна) і в жодному з наших запитів не потрібен. **Саме тому колонковий формат виграє:** Athena його просто не читає.

Data scanned і час виконання — фактичні заміри Athena

Запит	JSON (scanned / engine)	Parquet (scanned / engine)	менше
Q3 оборот	1 260 692 423 Б / 3 090 мс	**5 861 176 Б / 1 021 мс**	**215×**
Q4 відмови	1 260 692 423 Б / 4 271 мс	**667 929 Б / 1 017 мс**	**1 888×**
Q5 години	1 260 692 423 Б / 2 836 мс	**8 490 034 Б / 1 280 мс**	**148×**
`GROUP BY status`	1 260 692 423 Б / 3 420 мс	**129 022 Б / 1 167 мс**	**9 771×**

Час виконання впав лише у 2,2–4,2 раза — набагато менше за обсяг. Athena піднімає кластер і платить фіксовані накладні, тож під секунду запит не опускається незалежно від того, скільки даних читає.

Дві несподіванки, які дав реальний запуск:

1. **`count(*)` по Parquet сканує 0 байтів** — кількість рядків лежить у футері файлу. На JSON це неможливо: щоб порахувати рядки, треба їх прочитати.
2. **Partition pruning не працює крізь в'ю.** `SELECT count(*) FROM lab8.payments WHERE dt='2026-10-05'` просканував усі 1,17 GiB замість 1/31. Причина — `row_number() OVER (PARTITION BY invoiceId ...)` у в'ю: вікно розбиває дані за `invoiceId`, а не за `dt`, тож предикат по `dt` неможливо проштовхнути під віконну функцію. По Parquet-таблиці той самий запит сканує **0 Б за 413 мс**. Це ще один аргумент за крок 7: в'ю зручна, але вона скасовує партиціонування.

Але в грошах виграш менший, і це важливо сказати вголос. Athena тарифікує **мінімум 10 МБ на запит**:

	data scanned	білиться як	ціна при $5/TB
JSON	1 260 692 423 Б	1,17 GiB	**$0,006303**
Parquet	667 929 – 8 490 034 Б	10 MiB	**$0,00005**

Усі три Parquet-запити лежать нижче 10 МБ, тож коштують однаково. Реальна економія впирається в стелю ≈ **126×**, а не 1 888×. На 1,2 ГБ це $0,0063 проти $0,00005 — різниця без практичного значення. Parquet починає окупатися не на гігабайтах, а на терабайтах, і там ті самі 126× — це вже $6 300 проти $50 на місяць.

Є й другий, часто більший виграш, якого немає в таблиці: `lab8.payments` — це **в'ю**, тобто `DISTINCT *` + `row_number()` по 3 мільйонах рядків **перераховуються на кожному зверненні**. `payments_parquet` — матеріалізований результат тієї ж роботи. Різниця в часі виконання між ними більша, ніж різниця в data scanned, бо шафл для віконної функції — найдорожча частина запиту.

Повна таблиця замірів: results/benchmark.md, сирий лог із `QueryExecutionId`: results/benchmark.csv.

Шлях Б — Glue Crawler

```
./setup.sh crawler
aws glue get-crawler --name ooad-lab8-payments-crawler --query 'Crawler.State' --output text
aws glue get-table --database-name lab8_crawled --name payments \
    --query 'Table.{cols:StorageDescriptor.Columns, parts:PartitionKeys}' --output table
```

1. Які типи він обрав для `amount`, `ccy` і дат? Чи розпізнав `dt` як партицію?

Crawler відпрацював (`SUCCEEDED`, 31 партиція). Фактичний результат:

поле	наша схема	crawler	збіг
`amount`, `finalAmount`	`bigint`	**`int`**	❌
`ccy`, `mcc`	`int`	`int`	✅
`createdDate`, `modifiedDate`, `receivedAt`	`string`	`string`	✅
`dt`	партиція `string`	**партиція `string`**	✅
`failureReason`	`string`	`string`	✅
SerDe	`org.openx.data.jsonserde.JsonSerDe`	той самий	✅

`dt` розпізнано як партицію без жодних підказок — Hive-шаблон `dt=2026-10-01` у шляху є конвенцією, яку crawler читає сам. Дати лишилися рядками: ISO-8601 у `timestamp` він не перетворює, тож перевага ручної схеми тут нульова.

Єдина змістовна розбіжність — `int` замість `bigint` для грошей, і вона реальна:

```
max(amount) у датасеті :     8 004 389 копійок (80 043,89 ₴)
int32 max              : 2 147 483 647
запас                  : 268×
```

Поки запас 268-кратний, усе працює. Переповнення настане на платежі понад **21 474 836,47 ₴** — для B2B-еквайрингу це один великий рахунок. І тоді crawler за розкладом **мовчки змінить тип на `bigint`**, бо наступний семпл не влізе в int32; усе, що читало таблицю з очікуванням int, зламається — і не в момент деплою, а в момент надходження нетипового платежу. Ми поставили `bigint` одразу не тому, що вгадали, а тому що знаємо предметну область: **гроші в мінорних одиницях — завжди bigint**. Інференс цього знати не може.

Побічна знахідка: `Parameters.recordCount` у каталозі — **1 202 922**, тоді як `SELECT count(*)` повертає **3 030 197**. Помилка у 2,5 раза. Це не баг: crawler не рахує рядки, а оцінює їх за семплом. Але якщо взяти цю цифру з каталогу для планування — оцінка буде втричі занижена.

Повний вивід: results/crawler_schema.md.

2. Що сталося з `failureReason`, якого немає в більшості подій?

Він **є** у схемі як `string` — і стоїть **останнім у списку колонок**, після `receivedat`, хоча в JSON-об’єкті failure-події він іде перед `receivedAt`. Порядок колонок у crawler-а — це порядок

першої зустрічі ключа: решта 15 полів є в кожному рядку й потрапили в схему з першого ж об’єкта, а `failureReason` з’явився лише на першій failure-події.

У JSON Lines відсутній ключ — це не помилка і не NULL у файлі, його просто немає в об’єкті. JSON SerDe при читанні повертає для такої колонки `NULL`. Crawler сканує вибірку файлів, зустрічає хоч один `failure`-рядок із цим ключем — і додає колонку в схему. Оскільки в нас 6 % подій це failure (60 180 платежів), ймовірність не натрапити на жоден — нульова.

Але це щастя, а не гарантія. Якби поле зустрічалося в 0,01 % подій і лише в одному файлі з 31, crawler міг би його не побачити взагалі — він читає семпл, а не все. Тоді колонка просто **зникає зі схеми**, запити по ній падають з `Column 'failurereason' cannot be resolved`, і причину шукають годину. Це фундаментальне обмеження інференсу: **він бачить тільки те, що потрапило у вибірку.**

Інверсія тієї ж проблеми гірша: якщо в одному файлі поле приходить числом, а в іншому рядком, crawler або зводить тип до `string`, або створює **дві різні таблиці** для одного префікса, бо вирішить, що це різні схеми.

3. Яка схема зручніша для запитів і чому ручні схеми все одно поширені?

Для запитів — вони майже однакові, і це чесна відповідь. Crawler віддав робочу таблицю: `SELECT count(*) FROM lab8_crawled.payments` повертає ті самі 3 030 197. Ручна краща рівно в одному місці — `bigint` замість `int` — і це не «зручність», а коректність на даних, яких ще немає.

Чому ручні схеми все одно пишуть:

1. **Стабільність.** Crawler за розкладом може змінити тип колонки, і запит, який працював учора, сьогодні падає. Ручна схема в git не змінюється сама.
2. **Схема — це контракт, і він має проходити код-рев'ю.** DDL у репозиторії видно в PR: «чому тут double для грошей?» — питання ставлять до релізу, а не після. Інферована схема з’являється мовчки.
3. **Ціна й латентність.** Crawler — це окремий запуск, хвилини і гроші ($0,44 за DPU-годину). `MSCK REPAIR TABLE` або `ALTER TABLE ADD PARTITION` коштують нуль. Ще краще — **partition projection**: ви описуєте діапазон дат у `TBLPROPERTIES`, і Athena обчислює партиції на льоту, взагалі без звернень до каталогу. На таблиці з тисячами партицій це економить десятки секунд на кожному запиті.
4. **Контроль над тим, що читається.** Ручна схема може **ігнорувати** зайві поля. Продюсер додав десять нових ключів — ваша таблиця їх не бачить і нічого не ламається.
5. **Складні випадки інференс не тягне.** Вкладені структури, масиви, `struct<>` з опційними полями, поля, що міняють тип між версіями API, — усе це crawler або розвалює, або зводить до `string`.

Де crawler справді на місці: розвідка незнайомих даних («що взагалі лежить у цьому бакеті?»), дата-лейк із сотнями джерел, де вручну описати все нереально, і першого дня проєкту, коли треба швидко подивитися на дані. Типовий зрілий підхід — **crawler один раз для розвідки, далі згенерований DDL вичищається руками й кладеться в репозиторій**, а crawler вимикається.

Чому `DISTINCT` стоїть перед `row_number()` і що ламається без дедуплікації

Чому саме в такому порядку

Поширена відповідь — «без DISTINCT в'ю поверне дублікати» — **неправильна**, і на захисті це одразу видно. `row_number()` з `WHERE rn = 1` поверне один рядок навіть з двох ідентичних: він пронумерує їх 1 і 2 і другий відсіється. Два рядки повернув би `rank()` (обидва отримали б `rk = 1`), але ми користуємося `row_number()`.

Що справді втрачається:

1. Детермінізм. Який із двох однакових рядків стане першим — не визначено. Зараз вони байт-у-байт однакові, тож ніхто не помітить. Але як тільки ретрай почне відрізнятися хоч одним полем — переserialised float, пізніший `receivedAt`, збагачений `merchantName` — в'ю почне **віддавати різні дані на тих самих вхідних**, а Parquet-таблиця з кроку 7 — змінюватися між запусками. Баг — це невідтворюваний результат; дублікат — лише спосіб, яким він проникає.

2. `ORDER BY` з нічиєю робить вікно недетермінованим на рівні движка. Presto вільний взяти будь-який рядок, і вибір залежить від того, як лягли спліти, тобто від того, скільки файлів повернув S3 цього разу. Той самий запит, ті самі дані, інша відповідь наступного місяця.

3. Кількість дублікатів перестає бути вимірюваною. `count(*) - count(DISTINCT *)` — єдине чесне визначення «скільки подій передоставили». Якщо їх уже схлопнула інша операція, повідомити 0,997 % неможливо. А цю метрику треба моніторити: стрибок дублікатів з 1 % до 20 % — це зламаний ретрай на боці відправника, і дізнатися про це варто сьогодні, а не на квартальному звіті.

4. Це дві різні операції, що відповідають на два різні питання — і саме тому їх не можна зливати в одну:

	що прибирає	**природа**
`DISTINCT`	відправник сказав це двічі	**баг транспорту**, треба алертити
`row_number()`	платіж пройшов через статуси	**нормальний бізнес-факт**

Схлопнувши обидві в одну віконну функцію, ви приховуєте, що одна з них — помилка, на яку слід реагувати.

Що ламається без дедуплікації — з цифрами

(а) Підрахунок платежів. 3 030 197 замість 1 000 000 — **у 3,03× більше**. Ця помилка абсурдна настільки, що її помітять одразу. Безпечна.

(б) Частка відмов — ось це небезпечно. На сирих подіях кожен платіж додає в знаменник два не-failure рядки, і частка розмивається втричі:

paymentMethod / device	на сирих подіях	після дедуплікації
qr / android	2,24 %	6,71 %
card / android	2,01 %	6,04 %
card / ios	1,98 %	5,95 %
apple_pay / web	1,99 %	5,95 %

2 % замість 6 %. Число не виглядає зламаним — воно виглядає нормально. Жоден sanity-check не спрацює, ніхто не піде розбиратися, і продуктова команда півроку вважатиме, що з платежами все гаразд, поки реально відвалюється кожен шістнадцятий. Абсурдна помилка безпечна; правдоподібна — ні.

(в) Оборот — майже правильний, і тому найгірший. `finalAmount` дорівнює нулю в `created` і `processing`, тож історія статусів суму взагалі не роздуває. Подвоюються лише ~1 % передоставлених `success`-подій → **завищення виручки приблизно на 1 %**. Воно тихе, стабільне, не виходить за межі правдоподібного і спливе аж на реконсиляції з банком.

(г) Унікальні клієнти — не зачеплені взагалі. `COUNT(DISTINCT customerId)` ідемпотентний і дає однакову відповідь до і після дедуплікації. Саме тому «у мене запит відпрацював нормально» **не є доказом**, що пайплайн дедуплікований.

Загальне правило, яке варто сформулювати на захисті:

> Дедуплікація ламає не всі агрегати, а тільки ті, що **рахують події**. `COUNT`, `AVG`, частки — ламаються. `COUNT DISTINCT`, `MIN`, `MAX` — ні. `SUM` — залежить від того, чи нульове поле в неостаточних станах. Тому «працює» — не критерій, і дедуплікувати треба **до** агрегації завжди, а не там, де помітили розбіжність.

Результати запитів 2–6 (Athena, повний датасет)

Усі значення — фактичний вивід Athena. Статистика кожного запуску, разом із `QueryExecutionId`, — у benchmark.csv.

Запит 1 — перевірка таблиці

```
SELECT count(*) AS total_events FROM lab8.payment_events;
```

total_events
3 030 197

31 партиція (`count(DISTINCT dt)` = 31), scanned 1 260 692 423 Б.

Запит 2 — дедуплікація

```
SELECT count(*) AS payments, count(DISTINCT invoiceId) AS unique_invoices
FROM lab8.payments;
```

payments	unique_invoices
1 000 000	**1 000 000**

Скільки прибрав кожен крок:

	рядків
сирих подій	3 030 197
після `SELECT DISTINCT`	**3 000 000**
після `row_number() = 1`	**1 000 000**

`DISTINCT` прибрав **30 197** передоставлених копій (0,997 %), `row_number()` — **2 000 000** застарілих статусів. Рівно 3 події на платіж.

Запит 3 — оборот по мерчантах і валютах (тільки `success`)

30 рядків. Повна таблиця — q3_turnover.md. Гривневі:

merchantName	payments	turnover, ₴
Інтернет-магазин	87 115	310 259 259,21
АЗС	87 560	144 641 341,06
Квитки	87 303	77 749 863,65
Супермаркет	87 283	54 147 597,02
Доставка їжі	87 820	46 044 018,26
Аптека	87 374	38 554 151,04
Таксі	87 307	21 528 219,39
Онлайн-кінотеатр	87 357	13 016 193,00
Кав'ярня	87 637	11 435 687,68
Музичний сервіс	87 517	8 664 183,00

Кількість платежів однакова з точністю до 0,8 %, оборот відрізняється у 36 разів — усе вирішує середній чек (3 562 ₴ проти 99 ₴).

> **Увага на тип.** `CAST(sum(finalAmount) AS decimal(38,0)) / 100` повертає `310259259` — **без копійок**: Trino бере scale лівого операнда, і нуль обрізає дробову частину. `sum(finalAmount) / 100.0` дає `3.1025925921E8` (double, наукова нотація). Правильно — `CAST(... AS decimal(38,2)) / 100` → `310259259.21`.

Запит 4 — частка відмов по методу й пристрою

paymentMethod	device	payments	failures	failure rate
qr	android	6 675	448	**6,71 %**
apple_pay	android	83 677	5 130	6,13 %
google_pay	web	59 996	3 645	6,08 %
apple_pay	ios	83 164	5 025	6,04 %
card	android	183 332	11 079	6,04 %
google_pay	ios	59 761	3 590	6,01 %
card	web	183 435	11 020	6,01 %
qr	web	6 669	400	6,00 %
google_pay	android	59 450	3 563	5,99 %
apple_pay	web	83 505	4 966	5,95 %
card	ios	183 541	10 912	5,95 %
qr	ios	6 795	402	**5,92 %**

Різниці немає. Розкид 5,92–6,71 п.п. виглядає як сигнал, але для комірки на 6 675 платежів 95 % ДІ біноміальної частки при p = 0,06 становить **±0,57 п.п.** Верхній рядок

лежить на межі, решта — глибоко в шумі. Правильна відповідь на «який метод найгірший» — **жоден**.

Причини відмов теж рівномірні (~20 % кожна):

failureReason	n
Таймаут 3-D Secure	12 107
Перевищено ліміт	12 088
Відмова банку-емітента	12 025
Недостатньо коштів	12 006
Невірний CVV	11 954

Запит 5 — платежі по годинах доби (UTC, `createdDate`)

год	платежів		год	платежів
00	6 597		12	66 304
01	6 622		13	59 948
02	6 592		14	53 306
03	6 629		15	53 783
04	**6 585**		16	60 256
05	13 291		17	72 905
06	26 851		18	**80 338**
07	46 684		19	73 167
08	59 947		20	60 219
09	53 492		21	46 627
10	46 521		22	26 669
11	53 438		23	13 229

Нічна підлога ~6 600/год, обідній горб о 12:00 UTC, пік о **18:00 UTC (21:00 Київ)**. Відношення пік/провал — **12,2×**. Це й є причина, чому в архітектурі все на Fargate і Kinesis: залізо під пік простоювало б 2/3 доби.

Запит 6 — запізнілі події

total_events	3 030 197
late > 60 с	**60 824 (2,007 %)**
late > 1 год	**0 (0,0000 %)**
середнє запізнення	7,63 с
медіана	1 с
максимум	**600 с**

Розподіл бімодальний, а не плавний:

інтервал	**подій**	**%**
0–10 с	2 969 260	97,989
10–60 с	113	0,004
60–120 с	6 735	0,222
2–5 хв	20 137	0,665
5–10 хв	33 952	1,120

Покриття за grace period:

grace	**0 с**	**30 с**	**60 с**	**120 с**	**300 с**	**600 с**
покриття	32,65 %	97,99 %	97,99 %	**98,22 %**	98,88 %	100,00 %

Чим партиціоновано

```
подій, де день modifiedDate ≠ dt :  0
подій, де день receivedAt    ≠ dt : 88
```

`dt` — **день події**, не день отримання. 88 подій сталися перед північчю, а доїхали після. Це правильний вибір: партиція за добу закривається назавжди і денний звіт ніколи не змінюється заднім числом.

Порівняйте з частиною 4, де Firehose партиціонує за `!{timestamp}` — часом прибуття — і весь жовтневий день лягає в одну партицію з датою запуску реплею.

Запит 3 — повний результат: оборот по мерчантах і валютах

Тільки `status = 'success'`, суми в мажорних одиницях (`finalAmount / 100`). Пораховано по всьому датасету (31 день).

merchantName	ccy	currency	payments	turnover	avg ticket
Інтернет-магазин	840	USD	3 725	13 840 942,83	3 715,69
Інтернет-магазин	978	EUR	2 840	10 084 500,30	3 550,88
Інтернет-магазин	980	UAH	87 115	310 259 259,21	3 561,70
АЗС	840	USD	3 600	5 856 481,33	1 626,80
АЗС	978	EUR	2 873	4 708 258,32	1 638,80
АЗС	980	UAH	87 560	144 641 341,06	1 651,91
Аптека	840	USD	3 727	1 659 126,96	445,16
Аптека	978	EUR	2 758	1 203 052,19	436,20
Аптека	980	UAH	87 374	38 554 151,04	441,25
Доставка їжі	840	USD	3 754	2 036 639,20	542,52
Доставка їжі	978	EUR	2 727	1 427 829,18	523,59
Доставка їжі	980	UAH	87 820	46 044 018,26	524,30
Кав'ярня	840	USD	3 819	505 697,22	132,42
Кав'ярня	978	EUR	2 802	365 038,15	130,28
Кав'ярня	980	UAH	87 637	11 435 687,68	130,49
Квитки	840	USD	3 876	3 602 313,89	929,39
Квитки	978	EUR	2 745	2 451 613,23	893,12
Квитки	980	UAH	87 303	77 749 863,65	890,57
Музичний сервіс	840	USD	3 762	372 438,00	99,00
Музичний сервіс	978	EUR	2 777	274 923,00	99,00
Музичний сервіс	980	UAH	87 517	8 664 183,00	99,00
Онлайн-кінотеатр	840	USD	3 688	549 512,00	149,00
Онлайн-кінотеатр	978	EUR	2 840	423 160,00	149,00
Онлайн-кінотеатр	980	UAH	87 357	13 016 193,00	149,00
Супермаркет	840	USD	3 776	2 310 115,59	611,79
Супермаркет	978	EUR	2 841	1 737 547,31	611,60
Супермаркет	980	UAH	87 283	54 147 597,02	620,37
Таксі	840	USD	3 742	923 507,17	246,80
Таксі	978	EUR	2 875	724 136,69	251,87
Таксі	980	UAH	87 307	21 528 219,39	246,58

Що видно

Трафік у всіх однаковий, різниця — в чеку. Кількість успішних гривневих платежів у всіх десяти мерчантів лежить у вузькому коридорі 87 115–87 820 (розкид 0,8 %). Оборот

при цьому відрізняється у **36 разів** — від 8,7 млн ₴ у музичного сервісу до 310 млн ₴ в інтернет-магазину. Весь рейтинг визначає середній чек, а не популярність.

Підписочні сервіси видно за нульовою дисперсією. У «Музичного сервісу» середній чек рівно 99,00 у всіх трьох валютах, в «Онлайн-кінотеатру» — рівно 149,00. Це фіксована ціна підписки: кожен платіж на ту саму суму. У решти мерчантів середній чек плаває між валютами на одиниці відсотка — ознака випадкових сум.

Валютний розподіл стабільний: ~93,6 % платежів у UAH, ~4 % USD, ~3 % EUR у кожного мерчанта. Це теж артефакт генератора, але він зручний: будь-яке відхилення в окремого мерчанта одразу було б видно.

part2-batch-athena/results/benchmark.md

Data scanned і час виконання: JSON проти Parquet

Усі числа — фактичні, з `QueryExecution.Statistics` Athena (`DataScannedInBytes`, `EngineExecutionTimeInMillis`). Сирий лог — у benchmark.csv, кожен рядок містить `QueryExecutionId` для звірки в консолі.

Датасет: 3 030 197 подій, 1 260 672 849 Б JSON → 1 000 000 платежів, 88 309 777 Б Parquet.

Запити 3–5

Запит	Джерело	Data scanned	Engine time	Виграш за обсягом
Q3 оборот	`lab8.payments` (JSON view)	1 260 692 423 Б (1,17 GiB)	3 090 мс	
Q3 оборот	`lab8.payments_parquet`	**5 861 176 Б (5,6 MiB)**	**1 021 мс**	**215×**
Q4 відмови	`lab8.payments` (JSON view)	1 260 692 423 Б	4 271 мс	
Q4 відмови	`lab8.payments_parquet`	**667 929 Б (652 KiB)**	**1 017 мс**	**1 888×**
Q5 години	`lab8.payments` (JSON view)	1 260 692 423 Б	2 836 мс	
Q5 години	`lab8.payments_parquet`	**8 490 034 Б (8,1 MiB)**	**1 280 мс**	**148×**

Час виконання впав лише у 2,2–4,2 раза — значно менше, ніж обсяг. Причина в тому, що на 1,2 ГБ Athena все одно піднімає кластер і платить фіксовані накладні: під секунду запит не опускається, скільки б даних не було.

Column pruning і partition pruning — це різні речі

Запит	Джерело	Data scanned	Engine	Коментар
`GROUP BY status`	JSON	1 260 692 423 Б	3 420 мс	читає все, щоб знайти одне поле
`GROUP BY status`	Parquet	**129 022 Б (126 KiB)**	1 167 мс	**9 771×** — одна колонка з 17
`count(*) WHERE dt='2026-10-05'`	JSON	1 260 692 423 Б	3 657 мс	**партиція не спрацювала**
`count(*) WHERE dt='2026-10-05'`	Parquet	**0 Б**	**413 мс**	відповідь із метаданих
`count(*) WHERE date_format(createdDate…)`	Parquet	8 490 034 Б	1 608 мс	фільтр не по партиції → full scan

Дві знахідки, які варто пояснити на захисті

1. `count(*)` по Parquet сканує 0 байтів. Кількість рядків лежить у футері Parquet-файлу, тож Athena відповідає, не читаючи жодної колонки. На JSON це неможливо в принципі: щоб порахувати рядки, треба їх прочитати.

2. Partition pruning НЕ працює крізь в'ю `lab8.payments`. Запит `WHERE dt='2026-10-05'` просканував усі 1,17 GiB замість 1/31. Це не помилка Athena — це наслідок того, що в'ю містить `row_number() OVER (PARTITION BY invoiceId ...)`. Вікно розбиває дані за `invoiceId`, а не за `dt`, тому предикат по `dt` **не можна проштовхнути під віконну функцію**: платіж, створений 4 жовтня і завершений 5-го, має події в обох партиціях, і щоб коректно вибрати останню, движок зобов'язаний прочитати обидві.

Практичний висновок: **в'ю зручна, але вона скасовує партиціонування.** Саме тому крок 7 (CTAS у Parquet) — не косметика. Матеріалізована таблиця партиціонована і фільтрується нормально; в'ю — ні.

Скільки разів дешевше стало

	JSON	Parquet
Розмір на S3	1 260 672 849 Б	88 309 777 Б (**14,3×**)
Файлів	31	213
Рядків	3 030 197	1 000 000
Data scanned (Q4)	1 260 692 423 Б	667 929 Б (**1 888×**)
Ціна за $5/ТБ	$0,006303	$0,0000033
Фактична ціна	**$0,006303**	**$0,00005**
		≈126× дешевше

Головне застереження. Athena тарифікує **мінімум 10 МБ на запит**. Усі три Parquet-запити (0,65 / 5,6 / 8,1 MiB) лежать нижче цієї межі, тож усі три коштують однаково — $0,00005. Тому виграш у 1 888× за обсягом перетворюється на **126× у грошах**, і це стеля для цього датасету. На 1,2 ГБ різниця $0,0063 проти $0,00005 не має жодного практичного значення; Parquet починає окупатися на терабайтах, де ті самі 126× — це вже $6 300 проти $50 на місяць.

Чому Parquet від Athena більший за локальний

Локальний CTAS через DuckDB дав 79 080 980 Б у 31 файлі; Athena — 88 309 777 Б у **213 файлах**, на 11,7 % більше. Причина — паралелізм: Athena пише по файлу на воркер, тож на партицію виходить 6–7 файлів замість одного. Менші файли → менші row group-и → гірший словник і гірше стиснення. Це мініатюра **small files problem**: на 213 файлах вона коштує 11 % місця, на 200 000 файлів вона коштує більше, ніж самі дані.

Де справді гарячі байти

Розмір по колонках (локальний Parquet, сума по всіх партиціях):

колонка	MiB	частка
`reference`	30,71	**41 %**
`invoiceId`	10,49	14 %
`createdDate` / `receivedAt` / `modifiedDate`	по 6,22	25 %
`customerId`	5,43	7 %
`amount` / `finalAmount`	3,94 / 3,82	10 %
`merchantName` / `merchantId` / `mcc`	по 0,49	2 %
`paymentMethod` / `device`	по 0,25	0,7 %
`ccy` / `failureReason` / `status`	0,09–0,12	0,4 %

41 % файлу — `reference`, випадковий UUID без дефісів. Ентропія максимальна, тож він не стискається, і при цьому не потрібен жодному з наших запитів. Саме тому Q4 (`status`, `paymentMethod`, `device` — разом 0,45 MiB у локальному вимірі) сканує в 1 888 разів менше за JSON: **колонковий формат просто не читає 41 % файлу.**

Шлях Б — схема, яку вивів Glue Crawler

```
./setup.sh crawler
aws glue get-table --database-name lab8_crawled --name payments \
    --query 'Table.{columns:StorageDescriptor.Columns,partitionKeys:PartitionKeys,
                    serde:StorageDescriptor.SerdeInfo.SerializationLibrary,
                    recordCount:Parameters.recordCount,objectCount:Parameters.objectCount}'
```

Результат: `State: READY`, `LastCrawl.Status: SUCCEEDED`, зареєстровано **31 партицію**.

Порівняння схем

Поле	Наша (01_create_table.sql)	Crawler	Збіг
`invoiceid`	string	string	✅
`status`	string	string	✅
`failurereason`	string	string	✅ (але див. нижче)
`amount`	**bigint**	**int**	❌
`finalamount`	**bigint**	**int**	❌
`ccy`	int	int	✅
`reference`	string	string	✅
`createddate`	string	string	✅
`modifieddate`	string	string	✅
`receivedat`	string	string	✅
`merchantid`	string	string	✅
`merchantname`	string	string	✅
`mcc`	int	int	✅
`customerid`	string	string	✅
`paymentmethod`	string	string	✅
`device`	string	string	✅
`dt`	партиція string	**партиція string**	✅

SerDe: `org.openx.data.jsonserde.JsonSerDe` — **той самий**, що ми написали руками. `classification: json`, `objectCount: 31`.

---

1. Типи для `amount`, `ccy` і дат; чи розпізнав `dt`

`dt` — так, розпізнав як партицію типу `string`, без жодних підказок: Hive-шаблон `dt=2026-10-01` у шляху — це конвенція, яку crawler читає автоматично. Усі 31 партиція зареєстровані.

Дати — `string`. Crawler **не** намагається розпізнати ISO-8601 як `timestamp`. Тобто перевага ручної схеми тут нульова, але й програшу немає: обидві вимагають `from_iso8601_timestamp()` у кожному запиті.

`amount` і `finalAmount` — `int`, а не `bigint`. Це єдина змістовна розбіжність, і вона реальна:

```
max(amount) у датасеті :     8 004 389 копійок  (80 043,89 ₴)
int32 max              : 2 147 483 647
запас                  : 268×
```

Поки запас 268-кратний, усе працює. Переповнення настане на платежі понад **21 474 836,47 ₴** — для B2B-еквайрингу це один великий рахунок. І тоді станеться найгірше з можливого: crawler за розкладом **мовчки змінить тип на `bigint`**, бо наступний семпл не влізе в int32. Усе, що читало таблицю з очікуванням int, зламається або почне віддавати NULL — і зламається не в момент деплою, а в момент надходження нетипового платежу.

Ми поставили `bigint` одразу не тому, що вгадали, а тому що знаємо предметну область: **гроші в мінорних одиницях — завжди bigint**. Інференс цього знати не може, він бачить лише вибірку.

Побічна знахідка: `recordCount` у crawler-а неправильний

```
Parameters.recordCount : 1 202 922
SELECT count(*)        : 3 030 197
```

Помилка у **2,5 раза**. Це не баг: crawler не рахує рядки, він оцінює їх за семплом. Але якщо хтось візьме цю цифру з каталогу для планування — отримає втричі занижену оцінку. Реальна кількість доступна лише запитом.

---

2. Що сталося з `failureReason`

Він **є** у схемі як `string` — і стоїть **останнім у списку колонок**, після `receivedat`, хоча в JSON-об’єкті failure-події він іде перед `receivedAt`. Порядок колонок у crawler-а — це порядок, у якому він уперше зустрів ключ: решта 15 полів є в кожному рядку й потрапили в схему з першого ж об’єкта, а `failureReason` з’явився лише тоді, коли трапилася перша failure-подія.

У JSON Lines відсутній ключ — це не помилка і не `null` у файлі, його просто немає в об’єкті; JSON SerDe повертає для такої колонки `NULL`.

Але те, що він знайшовся, — це щастя, а не гарантія. 6 % подій у нас — failure (60 180 платежів, рівномірно по всіх 31 файлах), тож імовірність промахнутися нульова. Якби поле траплялося в 0,01 % подій і лише в одному файлі з 31, crawler міг би його **не побачити взагалі**: він читає семпл, а не все. Тоді колонка просто зникає зі схеми, запити падають з `Column 'failurereason' cannot be resolved`, і причину шукають годину.

Це фундаментальне обмеження інференсу: **він бачить тільки те, що потрапило у вибірку.** Інверсія проблеми гірша — якщо в одному файлі поле приходить числом, а в іншому рядком, crawler або зводить тип до `string`, або створює **дві різні таблиці** для одного префікса, вирішивши, що це різні схеми.

3. Яка схема зручніша і чому ручні все одно пишуть

Для запитів вони майже однакові, і це чесна відповідь. Crawler віддав робочу таблицю: `SELECT count(*) FROM lab8_crawled.payments` повертає ті самі 3 030 197. Ручна краща рівно в одному місці — `bigint` замість `int` — і це не «зручність», а коректність на даних, яких ще немає.

Чому ручні схеми все одно поширені:

1. **Стабільність.** Crawler за розкладом може змінити тип колонки, і запит, що працював учора, сьогодні падає. DDL у git не змінюється сам.
2. **Схема — це контракт, і він має проходити код-рев'ю.** DDL у репозиторії видно в PR: «чому тут double для грошей?» питають до релізу. Інферована схема з’являється мовчки.
3. **Ціна й латентність.** Crawler — окремий запуск, хвилини і гроші ($0,44 за DPU-годину). `MSCK REPAIR TABLE` коштує нуль. Ще краще — **partition projection**: діапазон дат описується в `TBLPROPERTIES`, і Athena обчислює партиції на льоту, взагалі без звернень до каталогу.
4. **Контроль над тим, що читається.** Ручна схема може **ігнорувати** зайві поля: продюсер додав десять нових ключів — таблиця їх не бачить і нічого не ламається.
5. **Складні випадки інференс не тягне:** вкладені структури, масиви, `struct<>` з опційними полями, поля, що міняють тип між версіями API.

Де crawler справді на місці: розвідка незнайомих даних, дата-лейк із сотнями джерел, перший день проєкту. Типовий зрілий підхід — **crawler один раз для розвідки, далі згенерований DDL вичищається руками й кладеться в репозиторій**, а crawler вимикається.